SAGE-IDS: Evidence-Gated Self-Evolving Intrusion Detection via Closed-Loop Multi-Agent Adversarial Validation

Anand Rawat, Anandi Singh, and Ananya Agarwal

***Abstract*—Intrusion-detection systems (IDS) are typically evaluated once, offline, and updated by hand. Proposals to make detection "self-evolving" risk the opposite failure: an automated loop that attacks, retrains, and deploys without control, and that can be gamed by its own feedback. We present SAGE-IDS, a multi-agent system in which a detector may change *only* through an evidence-gated, operator-approved protocol. Five agents—monitoring, threat research, adversarial validation, remediation, and explanation—are driven by a persistent, bounded state machine. For every proposed change, acceptance criteria are *derived from a measured baseline, frozen, and cryptographically committed before any candidate is evaluated*; candidates are compared against the current detector on identical traffic with a paired exact test and gated on regression, false-positive, and service-impact bounds. Each adversarial trial records three outcomes separately—executed, security effect achieved, detected—so detection is never conflated with prevention. On a controlled laboratory with real loopback packet capture and live NVD/MITRE ATT&CK threat intelligence, the protocol (i) accepts changes that raise held-out detection F_1 from 0.54 to 0.97 at zero measured false positives, (ii) *rejects* a plausible but over-sensitive change, and (iii) resists a feedback-poisoning shortcut (naive self-training) that a naive design would adopt. A dedicated robustness study shows the accepted detectors are evadable by low-and-slow and distributed variants, which we report as an explicit bound. A locally hosted large language model is confined to a closed action set; on 30 live findings it agreed with the reviewed mapper 97% of the time and never exceeded that set. All artifacts, seeds, dataset hashes, and a hash-chained audit ledger are released.**

***Index Terms*—Intrusion detection, multi-agent systems, adversarial validation, closed-loop evaluation, machine-learning security, reproducibility, large language models.**

I. Introduction

NETWORK intrusion detection has been studied for decades, from signature engines such as Snort [1] and Bro/Zeek [2] to a large body of machine-learning (ML) detectors surveyed in [3]. Yet a persistent gap separates research from practice. Sommer and Paxson [4] argued that intrusion detection differs from other ML domains in the cost of errors, the semantic gap between anomalies and attacks, and the difficulty of sound evaluation; many of their cautions remain unaddressed. ML-NIDS research continues to lean on datasets—KDD'99 and its cleaned successor NSL-KDD [5], UNSW-NB15 [6], and CICIDS2017 [7]—that are known to contain duplicates, label noise, and feature-extraction faults [8], and on random train/test splits that overstate generalisation to unseen attacks, a bias formalised by Pendlebury *et al.* [9].

A second gap concerns *adaptation*. Detectors based on random forests [10], isolation forests [11], autoencoder ensembles [12], or deep sequence models [13] are usually trained once; when traffic drifts, they are retrained manually. Work on concept-drift-robust NIDS [14] updates models automatically, but automated adaptation introduces its own hazards: an attacker who influences the feedback can poison the model, and adaptation under adversarial pressure is exactly where ML is weakest [15], [16]. Meanwhile, offensive tooling has become highly automated—adversary emulation platforms such as CALDERA [17] generate attacks from the MITRE ATT&CK knowledge base [18]—and large language models (LLMs) are increasingly proposed for security operations [19], raising the question of how to use such automation *safely* inside a defensive loop.

The phrase "self-evolving defence" therefore names a real need and a real danger. The need is to close the loop: detect, validate against realistic attacks, improve, and re-validate. The danger is that a naive closed loop becomes an unaccountable one—exploiting, retraining, and deploying without evidence, review, or bounds, and confusing *detecting* an attack with *preventing* it. We argue that the contribution worth making is not another adaptive model but a *validation discipline*: a protocol under which an IDS may evolve only when the evidence meets criteria that were fixed in advance, and only with a human in the loop.

We present SAGE-IDS (Self-evolving cyber defence through closed-loop multi-agent Adversarial validation and adaptive intrusion detection). Its contributions are:

1) **An evidence-gated self-evolution protocol.** Acceptance criteria are derived from a measured baseline, frozen, and committed to a hash-chained ledger *before* any candidate is tested; candidates are compared on identical traffic using a paired exact test [20] with interval estimates [21], and activation requires operator approval. "Self-evolving" thus means controlled, versioned, auditable change.
2) **A tri-outcome adversarial ledger** that records, per trial and separately, whether an attack executed, achieved its security effect, and was detected—so a fix that prevents without detecting, or detects without preventing, is visible rather than hidden.

The authors are with the Department of Computer Science and Engineering. Project mentor: Mr. Ashutosh Pandey. Correspondence: anandrawat138a@gmail.com.

Code, data manifests, and the full audit ledger are available at https://github.com/AnandRawat138/SAGE_IDS (see Data Availability).

3) **A feedback-poisoning control.** Retraining uses only labels verified by construction; we quantify, as an ablation on identical data, how a naive self-training loop fails.
4) **A safety architecture for offensive-capable automation**: in-code target allowlists enforced at every connection, opt-in active testing, LLM outputs confined to a closed reviewed action set, and a tamper-evident ledger, with a documented threat model.
5) **An open, reproducible evaluation** on a real packet-capture laboratory, a live threat-intelligence run, an evasion study, an LLM-vs-deterministic study, a leakage-controlled CICIDS2017 benchmark, and multi-seed repetition, with every number traceable to a released artifact.

II. Related Work and Positioning

Signature and anomaly NIDS. Snort [1] and Bro/Zeek [2] established rule- and policy-based detection; ML-based anomaly detection is surveyed in [3], with representative systems using isolation forests [11], autoencoder ensembles [12], and deep sequence models [13]. These target *what* to detect; SAGE-IDS is orthogonal—it governs *when and how* a detector is allowed to change.

Evaluation methodology. Sommer and Paxson [4] catalogued why ML-NIDS evaluation is hard. Dataset critiques [5], [8] and the space/time-bias framework of TESSERACT [9] show that common protocols inflate results. We adopt dedup-before-split, train-only preprocessing, and an unseen-day split, and reproduce the optimism gap (Sec. V-E); we extend this discipline from offline datasets to an online closed loop with pre-committed acceptance criteria.

Adaptive and drift-robust NIDS. Online detectors [12], [13] and concept-drift-robust systems such as INSOMNIA [14] update models as traffic changes. They do not pre-register acceptance criteria, do not separate prevention from detection, and do not place a human gate before activation—precisely the controls SAGE-IDS adds.

Adversary emulation and LLM agents. CALDERA [17] automates ATT&CK-based attacks [18]; LLMs are increasingly applied to security tasks [19]. SAGE-IDS uses bounded, reviewed emulation inside a validation loop and confines the LLM to selecting from a closed, auditable action set, measuring rather than trusting it.

Adversarial ML. Evasion and poisoning [15], [16] motivate our verified-label constraint and our explicit evasion study (Sec. V-C).

Table I positions SAGE-IDS against representative lines of work along the axes that define our contribution.

III. System Design

A. Agents and coordinator

SAGE-IDS comprises five agents coordinated by a persistent, bounded state machine (Fig. 1). The *Monitoring* agent ingests telemetry, builds per-source feature windows, attaches ground-truth labels, and runs the active detector. The *Threat Research* agent retrieves vulnerability and technique intelligence (Sec. III-D). The *Adversarial Validation* agent executes reviewed scenarios against allowlisted laboratory targets. The *Remediation* agent proposes bounded defensive changes. The *Explanation* agent produces evidence-grounded reports. Transitions are role-checked (system vs. operator), recorded with their evidence, and appended to a SHA-256 hash-chained ledger; running phases are claimed with an atomic compare-and-set so no phase executes twice; retries and adaptation cycles are bounded; and stale runs time out on restart rather than silently resuming.

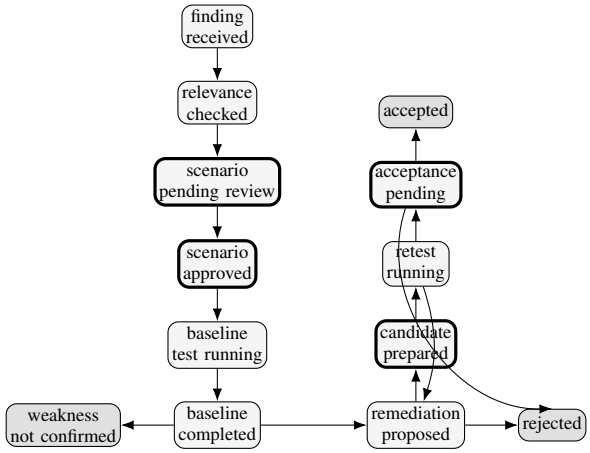


Fig. 1. Coordinator state machine (bold = operator-gated transition; shaded = terminal). Additional terminal states `not_relevant`, `inconclusive`, `failed`, `cancelled`, and `timed_out` are omitted for clarity. Each transition is validated, recorded with evidence, and hash-chained.

B. Laboratory and dual sensors

The laboratory consists of loopback services—a dummy login endpoint, a test web application, and decoy listeners—bound to localhost. Two sensors observe them. An application-level sensor records request/response metadata. A *real packet-capture sensor* (built on a user-space capture driver) sniffs loopback TCP, assembles bidirectional flows, and derives flow statistics in a CICIDS2017-compatible schema; unlike the application sensor it observes SYN/RST exchanges to closed ports, giving packet-faithful reconnaissance telemetry. An isolated container variant with Zeek and Suricata is provided as a deployment template.

C. Detection and versioning

A detector *version* bundles threshold rules, a random forest [10] (with an optional isolation forest [11] in shadow mode), the laboratory configuration, the feature-schema hash, and the label-set digest, all hashed together. Versions are candidate/accepted/retired; activation requires a human operator and a passed evaluation; rollback re-activates a previous accepted version. Rules are evaluated online over per-source sliding windows, yielding a detection latency as well as a decision.

D. Threat intelligence as untrusted data

The Threat Research agent retrieves CVE records from NVD and techniques from MITRE ATT&CK [18] over

TABLE I
POSITIONING OF SAGE-IDS AGAINST REPRESENTATIVE WORK. "CLOSED LOOP" = ATTACKS FEED BACK INTO VALIDATED DETECTOR CHANGE; "PRE-COMMITTED CRITERIA" = ACCEPTANCE THRESHOLDS FIXED BEFORE EVALUATING CANDIDATES; "PREVENTION ≠ DETECTION" = THE TWO OUTCOMES ARE MEASURED SEPARATELY; "POISONING CONTROL" = LEARNING RESTRICTED TO VERIFIED LABELS; "OPERATOR GATE" = HUMAN APPROVAL BEFORE ACTIVATION; "AUDIT" = TAMPER-EVIDENT PROVENANCE.

System / line of work	Closed loop	Adversarial validation	Pre-committed criteria	Prevention ≠ detection	Poisoning control	Operator gate	Tamper-evident audit
Signature IDS [1], [2]	–	–	–	–	n/a	manual	–
ML-NIDS surveys [3]	–	–	–	–	–	–	–
Online/anomaly [12], [13]	partial	–	–	–	–	–	–
Drift-robust (INSOMNIA) [14]	yes	–	–	–	partial	–	–
Adversary emulation [17]	–	yes	–	–	n/a	manual	–
Eval frameworks [9]	–	–	yes (offline)	–	–	n/a	–
SAGE-IDS (this work)	**yes**	**yes**	**yes**	**yes**	**yes**	**yes**	**yes**

Algorithm 1 One evidence-gated adaptation cycle

finding ← ThreatResearch(); scenario ← map(finding) ▷ closed set
require operator approval of scenario
$B \leftarrow$ BaselineTest(scenario, active version) ▷ exec/effect/detect per trial
if weakness not confirmed **then** report; **return**
$C \leftarrow$ DeriveCriteria(B); commit SHA-256(C) to ledger
for proposal in Remediation(B) **do**
  $v \leftarrow$ PrepareCandidate(proposal) ▷ verified labels only
  $R \leftarrow$ Retest(v, common seeds) ▷ paired with baseline
  if Evaluate(C, R) passes **and** operator accepts
    then activate(v); **break**
end for
Explain(evidence); **rollback** available

HTTPS, with host allowlisting, timeouts, caching, rate limiting, provenance capture, and deduplication. Retrieved text is treated as untrusted: it is sanitised, length-limited, scanned for instruction-like content, stored, and *never executed*. Findings are mapped to reviewed scenarios deterministically by weakness class (CWE) and technique. An optional LLM may corroborate the mapping but is confined to selecting identifiers from the closed allowed set (Sec. V-D).

E. Evidence-gated acceptance

After the baseline test, numeric criteria are derived from the measured baseline, frozen, and hash-committed before any candidate is evaluated. The objective depends on the confirmed weakness: a *detection* objective (paired McNemar gain [20], Wilson lower bound [21], and "effect not increased") or a *prevention* objective (effect-rate reduction, one-sided Fisher test, and "detection non-inferior"), plus a regression-non-inferiority bound over previously passed scenarios, a benign false-positive bound, and a service-health check. All candidates of a case are retested under common random seeds; the held-out final test uses a disjoint seed range. Algorithm 1 summarises one cycle.

IV. THREAT MODEL AND SAFETY

SAGE-IDS contains offensive-capable components, so its primary risk is being turned against unintended targets or being gamed through its feedback. We summarise controls; the full model is released.

Target confinement. A CIDR+port allowlist is enforced in code at scenario selection and again at every socket connection (hostname resolution, connected-peer re-check, source-range check); active scenarios are opt-in. *Untrusted intelligence.* Retrieved content is sanitised and never executed; the deterministic CWE/technique mapping, not the content, drives actions. *LLM confinement.* LLM output is schema-validated and intersected with the reviewed catalogue, so it can never widen the action set, change a target, or alter a state (verified empirically in Sec. V-D). *Unsafe-deployment prevention.* A candidate activates only after a passed evaluation against pre-committed criteria and explicit operator approval; there is no production path. *Integrity.* Model bundles carry SHA-256 digests checked on load; the audit ledger is hash-chained and verified. Residual risks—tamper-evident is not tamper-proof, loopback shares the host kernel, and the allowlist trusts the resolver—are documented.

V. EVALUATION

Unless noted, each experiment phase uses 30 trials, Wilson 95% intervals, and paired exact McNemar tests; attacker and benign clients use distinct loopback source addresses, giving ground-truth labels by construction. Environment: Python 3.12, scikit-learn 1.9, a 16-core host. Every figure and table is regenerated from a released JSON artifact.

A. Closed-loop case study

Table II lists the outcomes of one clean run. Three cases exercise the protocol: a detection weakness that is fixed (password guessing), a detection weakness fixed by retraining (request flood), and a *prevention* weakness that is correctly *not* fixed by detection (reconnaissance). A fourth finding carrying instruction-like injected text is flagged untrusted and denied by the operator.

For password guessing, a maximum-sensitivity rule is *rejected* because it raises the benign false-positive rate to 0.167 against the frozen bound of 0.02; a calibrated rule plus account lockout is accepted. For reconnaissance, the baseline already detects every probe ($30/30$) but the probe still achieves its

TABLE II
CLOSED-LOOP CASE OUTCOMES (RUN 145253Z). "BASELINE" GIVES PER-TRIAL DETECTION AND SECURITY-EFFECT OF THE INITIAL DETECTOR V1.

Case	Baseline (detect / effect)	Candidates	Outcome
Password guessing	0/30 / 30/30	2 (1 rejected)	accepted v
Request flood	0/30 / 30/30	1	accepted v
Reconnaissance	30/30 / 21/30	2	all rejecte
Injected finding	n/a	0	operator-d

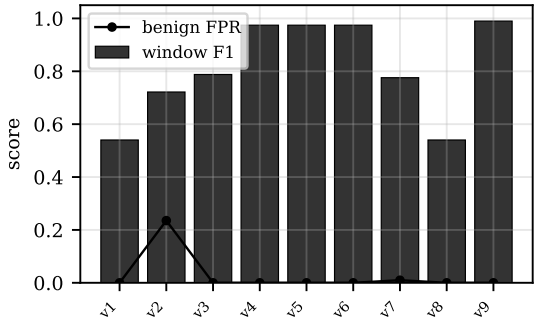


Fig. 2. Held-out window-level F_1 (bars) and benign false-positive rate (line) for each detector version in run 145253Z. Rejected candidates (e.g. the high-FPR v2) never activate.

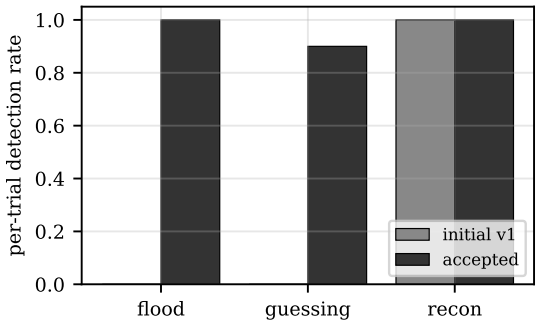


Fig. 3. Per-scenario held-out detection rate: initial detector v1 vs. accepted versions, identical traffic. Reconnaissance is already detected by v1; the gain is on guessing and flood.

TABLE III
CANDIDATE EVALUATIONS AGAINST PRE-COMMITTED CRITERIA (RUN 145253Z). ONLY PASSING CANDIDATES WITH OPERATOR APPROVAL ARE ACTIVATED.

Candidate	Objective	Result (binding check)
v2 (max-sensitivity rule)	detection	**reject** (benign FPR $0.167 > 0.02$)
v3 (calibrated rule+lockout)	detection	**pass**
v4 (verified retrain)	detection	**pass**
v5 (recon rule)	prevention	**reject** (effect not reduced)
v6 (recon retrain)	prevention	**reject** (effect not reduced)

effect ($21/30$); no detection-only candidate can satisfy a prevention objective, so both candidates are rejected—a decision the tri-outcome ledger makes principled rather than arbitrary. Fig. 2 shows window-level F_1 and benign false-positive rate across all versions produced in the run; Fig. 3 shows per-scenario detection of the initial vs. accepted detectors on the held-out final test (fresh seeds, identical traffic): window F_1 rises from 0.54 to 0.97 at $0/102$ benign false positives, flood detection from $0/30$ to $30/30$, and password-guessing from $0/30$ to $27/30$ (the three misses succeed within ≤ 3 attempts and are behaviourally indistinguishable from user typos).

Table III shows the frozen acceptance criteria and the pass/fail of each candidate, read from the committed evaluation records. The screenshot in Fig. 4 shows the operator dashboard exposing the committed criteria hash—the commitment a reviewer can verify against the ledger.

B. Feedback-poisoning and static-baseline ablations

Two ablations use the same held-out traffic. *Naive self-training*—retraining on pseudo-labels from the current model—detects $0/30$ of both attacks, because a model cannot label attacks it already misses; verified-label retraining detects $30/30$ of each. This is a direct, measured argument for the verified-label constraint. *Static baseline*—enabling the isolation forest on v1 with no adaptation—reaches window F_1 0.78; we therefore report the adapted version's gain over this stronger baseline (to 0.97), not only over the initial rules.

C. Robustness to evasion

We run reviewed evasive variants against the *accepted* detectors (Fig. 5). The accepted password-guessing detector falls from $27/30$ on the normal attack to $3/12$ (low-and-slow) and $0/12$ (distributed, one source per attempt); the flood detector catches $9/12$ of a low-rate flood. The accepted fixes are genuine improvements against the tested threat but are *not* robust defences—an honest bound that the protocol surfaces rather than conceals, consistent with the adversarial-ML literature [15].

D. Live threat intelligence and LLM confinement

A live run retrieved 46 findings (40 NVD CVEs, 6 ATT&CK techniques); 42 mapped to laboratory assets by weakness class, yielding real cases from current intelligence. We then measured a locally hosted LLM (Llama-3-8B) against the reviewed deterministic mapper. On 30 live findings the LLM produced valid structured output 100% of the time, independently selected the reviewed scenario 97% of the time at a mean latency of 2.75 s, and—by construction—*never* selected outside the closed allowed set. The LLM thus adds corroboration without widening the action space; even at 0% agreement the system remains safe, because only the deterministic mapping and the operator drive execution.

E. Benchmark with leakage controls

To place the detector family in context we benchmark on the full CICIDS2017 dataset [7] ($2{,}830{,}743$ flows; $2{,}496{,}544$ after dedup-before-split; zero cross-split identical-row overlap on every seed), with train-only preprocessing and validation-selected thresholds, over five seeds. Fig. 6 contrasts a random stratified split (random forest F_1 0.998) with an unseen-day split (train Mon–Thu, test Fri; F_1 0.540, recall 0.370). The

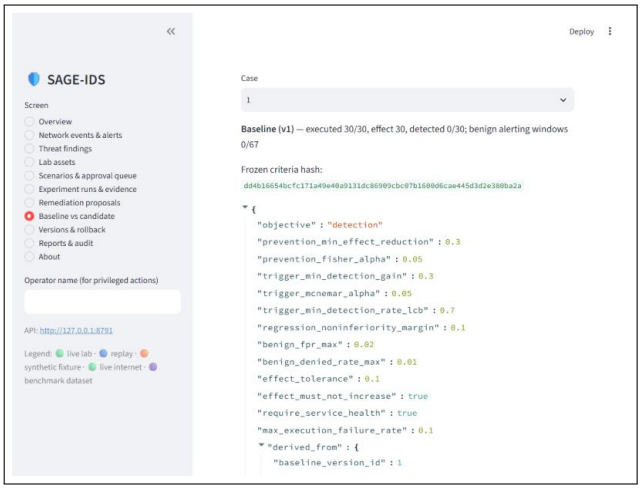


Fig. 4. Operator dashboard (baseline-vs-candidate screen) showing the baseline measurement and the *frozen criteria hash* committed before any candidate was evaluated.

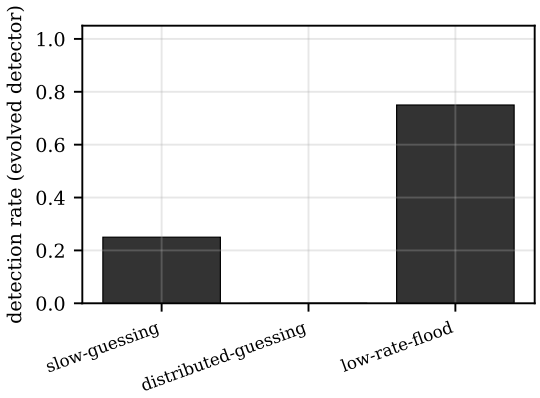


Fig. 5. Detection rate of the evolved detector against reviewed evasive variants. Each variant still achieves its security effect; low detection indicates evadability of behavioural per-source thresholds.

gap—more than 0.45 F_1—quantifies the optimism of random-split evaluation for unseen attack families, reproducing the space/time-bias concern [9] on a network dataset and motivating the unseen-attack posture of our closed loop.

F. Multi-seed stability

To address single-realisation bias we repeat the full protocol under three independent base seeds (attacker parameters, trial order, benign traffic, and model seeds all vary; Table IV). The accept/reject *decisions are identical across all three seeds*—password guessing and request flood accepted, reconnaissance rejected—and the held-out detection rates are tight: flood $0.00 \to 1.00$, password guessing $0.00 \to 0.90$, reconnaissance $1.00 \to 1.00$ (mean over seeds, 95% bootstrap interval width ≤ 0.00 at this precision). The protocol's outcomes are thus not artifacts of one random draw. The automated timing-integrity check flagged one seed in which two reconnaissance baseline trials ran abnormally long ($159\,\mathrm{s}$ and $317\,\mathrm{s}$) under host CPU contention (no inter-run gaps, i.e. not a host suspend); this inflates detection-latency on those two trials but does not affect

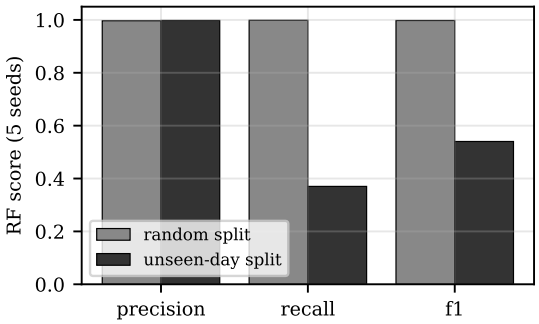


Fig. 6. CICIDS2017 random-forest performance (5 seeds): random split vs. unseen-day split. High recall collapses on attack families absent from training.

TABLE IV
MULTI-SEED STABILITY (3 REALISATIONS). CASE DECISIONS ARE IDENTICAL ACROSS SEEDS; HELD-OUT PER-SCENARIO DETECTION IS MEAN OVER SEEDS.

	seed 1	seed 2	seed 3
Password guessing	accept	accept	accept
Request flood	accept	accept	accept
Reconnaissance	reject	reject	reject
	detection: initial v1 $\to$ accepted (mean)		
flood	$0.00 \to 1.00$		
password guessing	$0.00 \to 0.90$		
reconnaissance	$1.00 \to 1.00$		

the logical executed/effect/detected outcomes or the decisions, which match the other seeds exactly. We report it rather than suppress it, as an instance of the integrity check working.

G. Cost and auditability

Detector inference costs on the order of $1\,\mathrm{ms}$ per feature window; one full closed-loop run (bootstrap, three cases, ablations, held-out test) completes in under 20 minutes on the 16-core host at $\approx 250\,\mathrm{MB}$ resident memory. The run's hash-chained ledger verifies intact, and a tamper test that modifies one ledger entry on a copy is detected at that entry—evidence integrity is mechanically checkable.

VI. DISCUSSION AND LIMITATIONS

SAGE-IDS's results are laboratory results. Targets are loopback services; benign traffic is synthetic; the scenario set is small. The operator is scripted in the reported runs, so the human gate is exercised mechanically rather than by independent judgement. The accepted fixes are evadable (Sec. V-C). The benchmark uses one dataset with known label issues [8]. The audit ledger is tamper-evident, not externally anchored. These bounds are stated so that the protocol's guarantees—pre-commitment, paired testing, tri-outcome accounting, verified-label learning, and operator approval—are not over-read. The design transfers directly to a packet-capture testbed with real services, which we provide as a container template; doing so, and adding evasive and multi-source scenarios to the approved catalogue, is the natural next step.

VII. Data Availability

The complete source code, configuration, synthetic fixtures, dataset SHA-256 manifests, model cards, per-run result JSON, generated reports, and the full hash-chained audit ledger are available at https://github.com/AnandRawat138/SAGE_IDS. The CICIDS2017 [7] and UNSW-NB15 [6] datasets are obtained from their original providers under their respective terms; we release only file hashes and preprocessing code, not the data. Every quantitative claim in this paper is reproduced by the commands in the repository's reproducibility guide.

VIII. Conclusion

SAGE-IDS reframes "self-evolving defence" as an auditable, evidence-gated protocol rather than autonomous adaptation. A detector may change only when pre-committed criteria are met on identical traffic under a paired test, learning is restricted to verified labels, and an operator approves the change; detection and prevention are measured separately throughout. On a real packet-capture laboratory with live intelligence, the protocol accepts genuine improvements, rejects an over-sensitive change and a mis-targeted one, resists a feedback-poisoning shortcut, and reports its own evadability. The contribution is the validation discipline and an honest, reproducible measurement of what such a loop does and does not achieve.

Acknowledgment

The authors thank their project mentor for guidance. This work used only locally hosted models and public threat-intelligence feeds.

References

[1] M. Roesch, "Snort: Lightweight intrusion detection for networks," in *Proc. 13th USENIX Conf. System Administration (LISA)*. USENIX Association, 1999, pp. 229–238.

[2] V. Paxson, "Bro: A system for detecting network intruders in real-time," *Computer Networks*, vol. 31, no. 23–24, pp. 2435–2463, 1999.

[3] A. L. Buczak and E. Guven, "A survey of data mining and machine learning methods for cyber security intrusion detection," *IEEE Communications Surveys & Tutorials*, vol. 18, no. 2, pp. 1153–1176, 2016.

[4] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *Proc. IEEE Symp. Security and Privacy (S&P)*. IEEE, 2010, pp. 305–316.

[5] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the KDD CUP 99 data set," in *Proc. IEEE Symp. Computational Intelligence for Security and Defense Applications (CISDA)*. IEEE, 2009, pp. 1–6.

[6] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems," in *Proc. Military Communications and Information Systems Conf. (MilCIS)*. IEEE, 2015, pp. 1–6.

[7] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Information Systems Security and Privacy (ICISSP)*. SciTePress, 2018, pp. 108–116.

[8] G. Engelen, V. Rimmer, and W. Joosen, "Troubleshooting an intrusion detection dataset: The CICIDS2017 case study," in *Proc. IEEE Security and Privacy Workshops (SPW)*. IEEE, 2021, pp. 7–12.

[9] F. Pendlebury, F. Pierazzi, R. Jordaney, J. Kinder, and L. Cavallaro, "TESSERACT: Eliminating experimental bias in malware classification across space and time," in *Proc. 28th USENIX Security Symp.* USENIX Association, 2019, pp. 729–746.

[10] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[11] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. 8th IEEE Int. Conf. Data Mining (ICDM)*. IEEE, 2008, pp. 413–422.

[12] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An ensemble of autoencoders for online network intrusion detection," in *Proc. Network and Distributed System Security Symp. (NDSS)*, 2018.

[13] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly detection and diagnosis from system logs through deep learning," in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*. ACM, 2017, pp. 1285–1298.

[14] G. Andresini, F. Pendlebury, F. Pierazzi, C. Loglisci, A. Appice, and L. Cavallaro, "INSOMNIA: Towards concept-drift robustness in network intrusion detection," in *Proc. 14th ACM Workshop on Artificial Intelligence and Security (AISec)*. ACM, 2021, pp. 111–122.

[15] B. Biggio and F. Roli, "Wild patterns: Ten years after the rise of adversarial machine learning," *Pattern Recognition*, vol. 84, pp. 317–331, 2018.

[16] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," in *Proc. 3rd Int. Conf. Learning Representations (ICLR)*, 2015.

[17] A. Applebaum, D. Miller, B. Strom, C. Korban, and R. Wolf, "Intelligent, automated red team emulation," in *Proc. 32nd Annual Conf. Computer Security Applications (ACSAC)*. ACM, 2016, pp. 363–373.

[18] B. E. Strom, A. Applebaum, D. P. Miller, K. C. Nickels, A. G. Pennington, and C. B. Thomas, "MITRE ATT&CK: Design and philosophy," The MITRE Corporation, Tech. Rep. MP180360, Case 18-0944-11, 2018.

[19] J. Zhang, H. Bu, H. Wen, Y. Liu, H. Fei, R. Xi, L. Li, Y. Yang, H. Zhu, and D. Meng, "When LLMs meet cybersecurity: A systematic literature review," *Cybersecurity*, vol. 8, no. 1, pp. 1–41, 2025.

[20] T. G. Dietterich, "Approximate statistical tests for comparing supervised classification learning algorithms," *Neural Computation*, vol. 10, no. 7, pp. 1895–1923, 1998.

[21] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.